Static Count-One Ownership for In-Place Array Updates in Lean 4

Gavin Mendel-Gleason
Scidonia Limited
gavin@scidonia.ai

30 October 2026

Lean statements are quoted from the development; hypotheses are abbreviated where the text says so.

Abstract

Purely functional programming is attractive for its equational reasoning, but without in-place updates some operations become asymptotically more expensive: an array write that copies under sustained sharing can turn a linear traversal into a quadratic one. Lean 4 recovers in-place updates through a *dynamic* mechanism: the runtime updates an array in place exactly when its reference count is one, and copies otherwise. The branch is decided at run time, by the heap's current sharing, not by the program's type. This paper asks whether a static type discipline can arrange that the dynamic check succeeds by construction at every update it certifies. Linear and affine type systems answer this question for their host languages by changing the type system—linear arrows, uniqueness or ownership types, or a borrow checker. We present a lightweight alternative for Lean that leaves the core type system untouched: a checked ownership discipline over a fragment of Lean's compiler IR, with a model heap and counts, a decidable checker whose soundness is proved in Lean, strict recognisers for the emitted array-write prefixes, and a correspondence between the recognised LCNF and the model. We derive the temporary extraction pair's count accounting and the put-back's exclusive branch, build a concrete runtime model in which the exclusivity check cannot abort under the correspondence, and bind the result to the pinned toolchain with an artifact/IR/C gate and a differential corpus. The guarantee carries two accounting contracts—the environment owner count and the heap/runtime count fidelity—and a kernel countermodel shows that the raw invariant does not imply them. The boundary is explicit: the C runtime's semantics, replay-to-backend identity, the emitted recursive callee and the source correspondence remain assumptions or open obligations.

1 Introduction

Reference-counted functional languages such as Lean 4 and Koka update a shared array by copying it and leave the in-place branch to the runtime's exclusivity test [5, 16, 19]. The resulting cost is data-dependent and invisible in the type: the same source program can copy or not depending on how many references happen to exist at the write. A static discipline that guarantees the count is one at the write would make the in-place branch a property of the program rather than of its current heap, and would let a library or a compiler rule out the copy branch without a dynamic test.

Lean 4 has no linear, affine or uniqueness types, and this development does not add them. The distinction is not the absence of a kernel change but the verified composition: a count-one checker, the recognised emitted prefix, the extraction/restoration accounting and a runtime model, joined at the boundary the paper names. Linearity is a checked property of a recognised fragment under named contracts, not a typing judgement for every program.

1.1 The chain to an in-place write

The result is a chain of eight conditional steps. Read in order, they say why an accepted update is in place; each names the premise it uses and the control that shows the premise is necessary, and the sections that follow prove it.

1. **The entry state.** `EntryInv` gives unique ownership, client separation and root completeness, and `initEntryInv` establishes it for a fresh array. Premise: no client alias. Control: the alias context of Example 2 is refused.
2. **Static acceptance.** A write requires an owned handle of multiplicity one; `check_sound` and `checkN_sound_of_entryInv` turn acceptance plus `Inv` into an exclusive write in the model. Control: the held context with multiplicity two is refused.
3. **Recognition.** `execAST_recognized` and its nested counterpart make the recognition hypothesis load-bearing: the executed trace is the recognised prefix. Control: the token-preserving near misses in the mutation corpus are rejected.
4. **Correspondence.** The three theorems `project_sound`, `recognized_write_in_place` and `index_path_simulates_model` identify the emitted outcome with the model's `inPlaceOutcome`. Premise: the layout, agreement and entry facts, the finite-path `PathContract`, and the contracts A1–A12/N1–N12.
5. **Temporary handles.** `pair_accounting` shows the extraction pair returns the parent and child counts to their entry values; the no-free and retained-alias controls leave a count of two.
6. **Restoration.** `putBack_exclusive_of_wellFormed` derives the mutate branch from the counted boundary and the child-call boundary; the shared, persistent and retained-parent controls reach the copy branch or are excluded.
7. **Runtime model.** Under `Corr`, `checkExclusive_no_abort` and `setAt_inPlace_of_entryInv` make the check return and the set mutate; `prefix_abort_impossible` states the same at the emitted-semantics boundary. Controls: shared and persistent sources, and a dirty allocator cursor.
8. **Linked corpus.** The gate and the differential corpus observe the agreement of the primitives on the pinned toolchain; the closed-pure-array case is the negative control that shows the compiler can still copy outside the model.

The physical reading also carries the environment-side `OwnersAccounted` and the heap-side `CountFidelity`; `alias_not_accounted` shows why the raw invariant does not imply the first. The chain is conditional: the C semantics of the primitives, replay-to-backend identity, address identity, free and cascade, the emitted callee, the accounting contracts and the source correspondence are not part of it and are listed in Section 10. The controls are not footnotes: without each premise the corresponding step fails, which is why they are proved beside the positive results.

Closest prior work. Marc Huisinga's 2023 thesis [6] implements a static uniqueness analysis for a model of Lean's IR and studies its checker; soundness and compiler integration remain future work there. Lean's `Array.markLinear` [12] is a dynamic facility: it can copy an array to establish uniqueness and, with `LEAN_ABORT_ON_NONLINEAR`, abort on later nonlinear use. Translation validation [15] checks a particular compiler run; this paper does not validate arbitrary transformations. The advance here is the verified composition of the count-one checker, the recognised emitted prefix, the

extraction/restoration accounting and a runtime model, not a claim to the first static uniqueness analysis for Lean.

The paper makes six contributions. (i) A model heap and a decidable checker whose soundness is a Lean theorem (Claim 2); an entry invariant supplies the heap facts the checker cannot see (Claim 3). (ii) Strict recognisers for the emitted array-write prefixes, with structural rejection of token-preserving near misses. (iii) A correspondence from the recognised LCNF to the model (Claim 4 and Claim 6), including the finite nested path. (iv) A counted extraction/call/restoration argument (Claim 8 and Claim 9). (v) A concrete runtime model and an abort-impossibility theorem (Claim 10), bound to the pinned toolchain by a gate and a differential corpus. (vi) The environment-side and heap-side accounting contracts (`OwnersAccounted`, `CountFidelity`), with a kernel countermodel showing why the raw invariant does not imply them, and the composed flat theorem `principal_write`.

The discipline is a deep embedding: `Code` and `NCode` are explicit syntax interpreted by `run`/`runN`, and `Ctx`, `Heap`, `check` and the theorems are ordinary Lean definitions. The source is Lean, lowered to LCNF; a recognised prefix is mapped to a model program. The guarantee is conditional on the named contracts, including the environment-side accounting and the heap/runtime count fidelity, and it does not extend to the linked C runtime beyond the measured gate.

System	Guarantee	Enforced by	Integration boundary
Huisinga 2023 [6]	uniqueness for a model of Lean IR	checker in Lean	no compiler link; soundness future work
Lean `Array.markLinear` [12]	dynamic mark/abort	runtime flag	source annotation
Translation validation [15]	per-run transform validation	validator	outside the compiler
Linear Haskell [2]	linear arrows, multiplicities	type checker	language extension
Rust [8]	ownership and borrows	borrow checker	type system
Clean [1]	uniqueness types	type system	language
Alias/ownership types [18, 4, 7]	alias restriction	type system	language
Koka/Perceus [13, 16]	reference counting with reuse	compiler and runtime	compiler
This work	checked count one, recognised prefix	external checker, Lean proofs	source unchanged; LCNF proof; C assumed

Table 1: Guarantees and integration boundaries.

Table 1 separates the guarantees from the integration boundary. Linear Haskell, Clean, alias types, ownership types and Cyclone put linearity in the type system; Rust's borrow checker is part of the type system; Koka and Perceus rely on the compiler's count and reuse. Huisinga analyses a model of Lean's IR without a compiler link. This work checks the emitted fragment and proves the connection to a model and a runtime model, with the linked C runtime remaining a measured assumption.

2 Setting: Lean 4, LCNF, and reference counting

Lean 4 compiles through a sequence of IRs. The last LCNF phase inserts reference-count operations, and the backend emits C. The runtime decides an array update with `lean_is_exclusive`: count one and non-persistent means in place; every other case copies. The runtime is single-threaded; count one and non-persistence characterise this test in that setting and are not a claim about every Lean runtime mode. The development pins the toolchain, the sibling revision and the captured LCNF artifacts, so every claim names the code it is about. The problem is not to implement reference

counting, but to know statically that the count at a particular write is one. Lean's source type system has no linearity or uniqueness; adding one would be a language change. We therefore state the property over the compiler's code and prove the connection, rather than extend the type system.

2.1 Model programs

The checker does not run on Lean source or on LCNF directly. Its programs are two small inductive types, interpreted over the model heap and environment; the compiler connection is a mapping from the recognised LCNF to these values.

```
inductive LetF where
  | lit (n : Nat)
  | size (a : Var)
  | isShared (x : Var)
  | checkAlias (x : Var)

inductive Code where
  | let (x : Var) (v : LetF) (k : Code)
  | inc (x : Var) (n : Nat) (k : Code)
  | dec (x : Var) (n : Nat) (k : Code)
  | check (x : Var) (k : Code)
  | setArr (x : Var) (i : Var) (v : Var) (k : Code)
  | ret (x : Var)

inductive NCode where
  | base (c : Code)
  | takeOut (a i fst snd : Var) (e : Ref) (k : NCode)
  | putBack (a e i result : Var) (k : NCode)
  | callAt (state path x result : Var) (body : NCode) (k : NCode)
  | ret (x : Var)
```

A `Code` is the straight-line array-write fragment; an `NCode` adds extraction, restoration and the recursive call. Variables are `Nat`, references are `Nat`, `Val` is a reference, a scalar or erased, and a `Heap` maps references to objects with a field array, a count and a persistence flag. `run`/`runN` execute these values and `check`/`checkN` are the checkers; the soundness theorems are about this semantics. They are neither Lean source nor LCNF: `projectPrefix` and `projectNested` walk an LCNF value, `Prefix.toModel` and `indexModelTerm` read a `Code`/`NCode` off the recognised operands, and `execAST`/`execNested` execute the recognised trace. The correspondence theorems relate that execution to the model run.

2.2 A running example: the nested write

The source is the array case of `Layout5.setAt` in the sibling. It takes the element out, checks the extracted focus, recurses on it and puts the result back:

```
| _, .index i q, y, s =>
    if h : i < s.size then
      match takeOutAt (assertExclusiveArray openSite s) i h (stateDefault _) with
      | ⟨(sib, focus), hbound⟩ =>
        plugAt sib i hbound (setAt q y (assertExclusive structSite focus))
    else s
```

The emitted C for this arm is the sequence the development names: a `lean_is_exclusive` test for the array, the `takeOut` exchange (`lean_array_fget`, `lean_array_fset`, the pair allocation), the child check on the extracted element, the recursive call and the `lean_array_fset` put-back. The recogniser `projectNested` accepts the `Path.index` arm, `PairProjection` accepts the two pair arms, and `execNestedAST` executes the accepted trace. The model term is `Path.build path value body ls`; `checkN` checks it from `initCtx`.

The example separates what the tool computes from what a client supplies. The compiler computes the LCNF and the C; `projectNested` computes the prefix; `Prefix.toModel`/`indexModelTerm` compute the model program; `checkN` computes the checker result. The client supplies the source declaration, the `Path` and the take-out name, and the proofs of `Agrees`, `LevelMatch`, `PathContract` and `EntryInv` (the path/slot and entry facts the checker cannot see). The finite-path induction discharges the tail contract; the leaf contract and the emitted call's meaning (N10) remain. A nearby rejected variant is the flat `aliasSet` example below, refused because the array's multiplicity is two.

3 The ownership discipline

3.1 The judgement and its rules

A context records, for each variable, whether it is owned, borrowed, scalar or dead, which reference it denotes, and the multiplicity of that reference. The checker is a total function from a context and a program to a successor context or failure; the frame stack records open extractions. The rules in Table 2 are ownership obligations: a write is admitted only for an owned handle of multiplicity one, an extraction only with the frame discipline, and a call only with an isolated body that returns the state it was handed.

The object language is untyped: there is no type former for a program, and the checker does not infer one by unification. The typing judgement is $c \vdash \mathit{code} \Rightarrow c'$, defined to hold exactly when `check c code = some c'`; the context is the type of the program point. A context is a usage map, a reference map and a multiplicity map, and the checker is the syntax-directed decision procedure for the rules below.

```
inductive U where
  | owned
  | borrow
  | scalar
  | dead

structure Ctx where
  us    : Var → U
  refOf : Var → Option Ref
  mult  : Ref → Nat

structure Frame where
  arr  : Ref
  idx  : Var
  elem : Ref

def check   : Ctx → Code → Option Ctx
def checkNF : Ctx → List Frame → NCode → Option Ctx
```

The judgement is syntax-directed: each constructor has one rule, the context is an input, and the successor context is computed by `setUs`, `move`, `bindOwned`, `inc` and `dec`. There are no metavariables, no unification and no implicit type arguments in the object language; the only computation is the context update. This is why the discipline makes the count visible at the type level: at the write, the array's handle has usage `owned` and multiplicity one in the context, and the rule refuses every other usage.

Rule	Obligation	Effect
`let x (lit n)`	—	`x` scalar
`let x (size a)`, `let x (isShared a)`	`a` owned or borrow	`x` scalar
`let x (checkAlias a)`	`a` owned, multiplicity one	`x` is the checked alias; `a` dead
`check x`	`x` owned, multiplicity one	unchanged
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`inc x n`	`x` owned	multiplicity $+n$
`dec x n`	`x` owned; $n <$ multiplicity	multiplicity $-n$; `x` dead
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; `e` ≠ `a`; no open frame at `a`	push the frame; bind parent and element
`putBack a e i result`	`a`, `e` owned, multiplicity one; the frame is `(a,i,e)`; `result` dead	pop the frame; bind the result
`callAt state path x result body`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree body`; no open frame at `state`; the body is accepted from the isolated context and returns `state`	call the body; bind the result
`ret x`	no frame: `x` owned or scalar; one frame: `x` is the frame's element; deeper: refused	exit

Table 2: The checker's ownership obligations.

Two consequences of the table are worth stating. First, `dec` is affine, not the last drop: its premise is $n <$ the object's multiplicity, so it consumes a duplicate handle and leaves the object live. The last release would free the cell, which this fragment does not model; free and cascade are named limits. Second, linearity at the write is the side condition `multOf x = 1`, not the usage tag alone: `us x = owned` says the variable names the object, while the multiplicity says it is the only handle.

A derivation. The first example is the two rules applied in sequence. The premise of the second rule is the context the first produces; no type variable is solved.

$$\frac{\text{us } 0 = \text{owned} \qquad \text{multOf } 0 = 1 \qquad c_1 = c_0.\text{move } 0\ 3}{c_0 \vdash \texttt{checkAlias}\ 0 \Rightarrow c_1} \qquad \frac{\text{us } 3 = \text{owned} \qquad \text{multOf } 3 = 1 \qquad \text{us } 1 = \text{us } 2 = \text{scalar} \qquad c_1 \vdash \texttt{ret}\ 3 \Rightarrow c_1}{c_1 \vdash \texttt{setArr}\ 3\ 1\ 2;\ \texttt{ret}\ 3 \Rightarrow c_1}$$

Composing the two gives $c_0 \vdash \texttt{execSet}\ 0\ 3\ 1\ 2 \Rightarrow c_1$, i.e. `check (initCtx 0) (execSet 0 3 1 2) = some ((initCtx 0).move 0 3)`. The checker computes c_1; the derivation is the rule applications, and the accepted program is the one the first example's guard checks.

The frame stack is the local form of the ownership invariant. An extraction opens a frame naming the array, index and element; a put-back closes the matching frame; a call checks its body under an isolated context and continues with the result. The exit rule is deliberately conservative:

with one open frame only the extracted element may return, and a deeper stack must be restored first. These are the rules that make nested updates safe, and they are exactly the parts a source-level linear type system would encode in types; here they are a checker.

The checker sees the shape of the context, not the heap: slot contents, counts, client separation and root completeness are supplied by `Inv` and `EntryInv`, and by the fixture's heap.

3.2 Worked examples

Each example is a `Code` or `NCode` value; the `#guard` is the executable check, and the surrounding context and heap are the repository's fixtures. The first two isolate the write rule; the next two the frame stack; the last two the call's isolation.

Example 1 (A unique write).

```
def execSet (x y i v : Var) : Code :=
  .let y (.checkAlias x) (.setArr y i v (.ret y))

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
```

Example 2 (An aliased write is refused).

```
def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails; the same refusal appears at the context level for a heap whose count is two. This is the static counterpart of the runtime copy: the checker rejects the program before it can copy.

Example 3 (An extraction and its return).

```
def good : NCode :=
  .takeOut 0 1 2 3 1 (.ret 3)

#guard (checkN ctx0 good).isSome
```

Example 4 (A stale slot is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1 (.takeOut 2 1 4 5 1 (.ret 5))

#guard (checkN ctx0 bad).isNone
```

The second extraction reopens the array whose slot the first has already emptied. The frame stack records the first extraction, so the second is refused; the runtime would fault at that point, which the fixture also checks.

Example 5 (A recursive body confined to its state).

```
def good : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9 (.ret 3) (.ret 9))

#guard (checkN ctx0 good).isSome
```

Example 6 (A suspended parent is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9
      (.takeOut 2 4 5 6 2 (.putBack 5 6 4 8 (.ret 3)))
      (.ret 9))

#guard (checkN ctx0 bad).isNone
```

The refused body returns the call's own state, exactly as the accepted body does; the difference is isolation. The body runs from an isolated context in which the suspended parent is dead, so opening it is not a typing error in the host language but a failure of the checker's ownership obligation.

3.3 The invariant and soundness

The model heap, its counts and exclusivity are the semantic side of the judgement: `Inv` ties the context's multiplicity to the heap count, and `Exclusive` is count one and non-persistence. The checker's soundness theorem is the bridge from the static obligation to the heap: every accepted write selects the exclusive branch. The entry invariant adds the facts a local checker cannot see: the invariant tree, client separation and root completeness. For fresh arrays `initEntryInv` establishes it, and `checkN_sound_of_entryInv` combines it with the checker.

Claim 1 (Invariant and exclusivity).

```
structure Inv (H : Heap) (env : Env) (c : Ctx) : Prop where
  rc_eq         : ∀ r, H.rc r = c.mult r
  owned         : ∀ x r, c.us x = .owned → c.refOf x = some r →
                    env x = some (.ref r) ∧ 1 ≤ c.mult r
  borrow        : ∀ x r, c.us x = .borrow → c.refOf x = some r →
                    env x = some (.ref r) ∧ H.cells r ≠ none
  scalar        : ∀ x, c.us x = .scalar → ∃ n, env x = some (.scalar n)
  nonPersistent : ∀ r o, H.cells r = some o → o.persistent = false
```

```
def Exclusive (H : Heap) (r : Ref) : Prop :=
  ∃ o, H.cells r = some o ∧ o.rc = 1 ∧ o.persistent = false
```

The displayed invariant relates the heap count to the context multiplicity, but it does not count the environment's owning variables: `Inv.owned` only requires `1 ≤ c.mult r`. `bindOwned` preserves `Inv` without changing `mult`, so two owned variables can denote one count-one cell. The raw invariant therefore does not imply physical uniqueness. The kernel countermodel `alias_not_accounted` constructs exactly that state: `initCtx` has one owner, and binding a second owned variable leaves the multiplicity at one.

The guarantee therefore carries two accounting contracts. The environment side is `OwnersAccounted`: multiplicity zero has no owner, and multiplicity one has at most one owner. The heap side is `CountFidelity`: the object's count is its incoming heap edges plus the owning environment variables in the supplied support.

```
def OwnersAccounted (c : Ctx) : Prop :=
  (∀ r x, c.mult r = 0 → c.us x = .owned → c.refOf x = some r → False) ∧
  (∀ r, c.mult r = 1 → ∀ x y, c.us x = .owned → c.us y = .owned →
      c.refOf x = some r → c.refOf y = some r → x = y)

def IncomingCount (H : Heap) (r : Ref) : Nat :=
  ((List.range H.next).map fun q =>
    match H.cells q with
    | some o => (o.fields.toList.filter (fun v => v = .ref r)).length
    | none => 0).sum

def EnvOwnerCount (c : Ctx) (vars : List Var) (r : Ref) : Nat :=
  (vars.filter (fun x => c.us x = .owned ∧ c.refOf x = some r)).length

def CountFidelity (H : Heap) (c : Ctx) (vars : List Var) : Prop :=
  ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c vars r
```

`Ownership/Accounting.lean` proves preservation of `OwnersAccounted` under a non-owning `setUs`, `move` of an owned handle, `inc`, and `bindOwned`/`dec` with explicit preconditions: no existing owner for the target of `bindOwned`, and at most one other owner for `dec`. The checker's take-out and `dec` rules do not yet enforce those preconditions; the principal theorem carries them as client obligations. The heap-side `CountFidelity` is an implementation assumption, not a theorem about the C runtime.

Claim 2 (Checker soundness).

```
theorem check_sound {c : Code} {ctx ctx' : Ctx} {H : Heap} {env : Env}
    (hc : check ctx c = some ctx') (hi : Inv H env ctx) :
    ∃ H' env' v ws,
      run c env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' ctx'
```

Claim 3 (Entry invariant).

```
structure EntryInv (H : Heap) (env : Env) (c : Ctx) (root : Ref)
    (client : List Ref) : Prop extends Inv H env c where
  uniqueTree : UniqueTree H root
  separated  : ClientSeparated H root client
  complete   : RootsComplete c H root
```

```
theorem checkN_sound_of_entryInv
    (hEntry : EntryInv H env c root client)
    (hag : Agrees H env code) (hc : checkN c code = some c') :
    ∃ H' env' v ws,
      runN code env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' c'
```

The quoted signature is the repository's; the heap and environment parameters are implicit.

The checker's local rules do not name the heap's path structure. The path theory supplies it: agreement composes through take-out, call and put-back, annotation reachability follows the path, and the safety results lift the checker to every finite path. This is where the per-client proof obligations are replaced by structural facts.

4 Recognising the emitted write

A source-level discipline is only useful if it survives lowering. The compiler rewrites the program, inlines helpers and chooses the emitted shape; the recogniser therefore walks the actual LCNF value, pinning operation order, arities, operand identities and branch constructors. It rejects the token-preserving near misses in its mutation corpus (eleven at the top level, ten on the nested path, fourteen per pair arm), so the correspondence is about the code we name, not a look-alike. No universal rejection theorem is claimed. The accepted grammar fixes the check's argument, the array-set's array, index and value operands, the instruction order from check to set, the pair constructor and its two projections, the two `Path` branches and their join, and the continuation boundary; every other shape is rejected. Three shapes are recognised: the top-level `execP` write, the nested `setAt` path, and the two `Path.left`/`Path.right` pair arms with their joins. The pair arms are the test that the recogniser handles control flow structurally rather than flattening it; the path and the pair carry their own contracts (A1–A12 and N1–N12), named where they are assumed.

5 Correspondence to the ownership model

The checker is about the model, not about the compiler. The bridge executes the operation trace the recogniser emitted for the actual declaration and proves it equal to the model run; the recognition hypothesis is load-bearing because `execAST` returns nothing unless the code decodes to the expected prefix. The primitive meanings are the named contracts, not theorems about Lean's backend.

Claim 4 (Prefix correspondence).

```
theorem project_sound (assign : FVarId → Var) (p : Prefix)
    (code : Code .impure) (hproj : projectPrefix code = some p)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
      some (PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H))
```

The layout and the heap/environment parameters are implicit, as in the repository.

Claim 5 (The write is in place).

```
theorem recognized_write_in_place
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
    PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H) =
        inPlaceOutcome root i v o H
```

The machine, environment and heap are implicit.

`PrefixOutcome` *records the target, the result binder, the heap, the disposition, the fault and the write trace. It discards the machine's variable map and any unrecognised suffix. The write trace preserves each covered write's exclusive/copy bit, so a projection to final contents would lose the property and is not used. The recogniser and executor share the prefix representation;* `execAST_recognized` *is the decoding adequacy lemma, not an independent instruction semantics. The independent meanings are the primitive contracts A1–A12; the shared representation is why the decoding hypothesis is load-bearing.*

The nested path is where a finite model meets recursive code. The model nests take-out, call and put-back per level; the finite-path induction discharges the callee contract from the tail and leaves only the leaf contract and the emitted call's meaning (N10). The model's flat environment cannot express the emitted per-invocation `oproj[1]` *index; that bound is recorded rather than hidden.*

Claim 6 (Nested finite path).

```
theorem index_path_simulates_model
    (declName takeOut : Name) (assign : FVarId → Ownership.Var)
    (p : NestedPrefix) (code : Code .impure)
    (hproj : projectNested declName takeOut code = some p)
    (ls : List Level)
    (hpath : PathContract path value body tip l rest H mEnv c cb y) :
    NestedOutcome.ofExec assign p
        (execNestedAST declName takeOut assign body p code s) =
      NestedOutcome.ofModel (Nested.runN (build path value body ls) mEnv H) ∧
    ∃ H' env' ws,
      Nested.runN (build path value body ls) mEnv H =
        .done H' env' (.ref rA) ws ∧ (∀ w ∈ ws, w.2 = true)
```

The level facts, agreement and entry state are implicit; the leaf contract is the remaining supplied premise.

Claim 7 (The finite nest).

```
theorem calls_build (path value : Var) (body : NCode) (ls : List Level) :
    calls (build path value body ls) = calls body + ls.length
```

```
theorem callBody_build (path value : Var) (body : NCode) (l : Level)
    (rest : List Level) :
    callBody? (build path value body (l :: rest)) =
      some (build path value body rest)
```

6 Counted extraction, call, and restoration

The emitted take-out creates a pair whose fields are the parent and the extracted child. The constructor moves the handles into the pair; the projections increment them; freeing the unique pair releases both edges. The counts are therefore equal at the boundary, but only because the pair is freed: the negative controls show that omitting the free, or retaining an alias, leaves a count of two. Without this accounting, N2's neutrality would be a modeling premise rather than a derived fact.

Claim 8 (Pair accounting).

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧
      B.2.cells p ≠ none ∧ B.2.cells c ≠ none ∧
      B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

```
theorem no_free_components_not_exclusive
    (hw : WellFormedEntry H p c i) :
    let B := protocolNoFree H p c i
    B.2.rc p = 2 ∧ B.2.rc c = 2 ∧ ¬ Heap.Exclusive B.2 p
```

Stage	parent rc	child rc	pair	ownership
entry	1	1	absent	root and element owned
take-out constructor	1	1	holds both	fields moved; no new component owners
projections	2	2	live	temporary local handles
free unique pair	1	1	gone	both edges released
child call	1	1	suspended parent	child runs in the isolated context
put-back	1	1	gone	mutate branch; parent and child restored

Table 3: The counted protocol at the extraction boundary.

The take-out shape is the `Path.index` *arm recognised by* `projectNested`*; the left/right arms are recognised by* `PairProjection`*, which reads the model term off the operands. The execution correspondence between the*

recognised pair and `Ownership.Pair.runP` *is a named next step; the counted protocol above is the accounting model, not that correspondence.*

The put-back is the second place where a copy can hide. The operation has two dispositions: at count one it mutates the parent; at a higher count or a persistent parent it allocates a fresh cell and decrements the input. The exclusive branch is derived from the counted-pair boundary and the child-call boundary, with functional restoration and count preservation separate; the returned-handle accounting remains a named premise.

Claim 9 (Put-back exclusivity).

```
theorem putBack_exclusive_of_wellFormed
    (hw : CountedPair.WellFormedEntry H p c i)
    (hc : PutBack.ChildCall (protocol H p c i).snd Hc env ctx state path x body p c) :
    ∃ op,
      putBack Hc p i c = some (p, mutate Hc p (parentAfter op i) i c) ∧
      (mutate Hc p (parentAfter op i) i c).rc p = 1 ∧
      (mutate Hc p (parentAfter op i) i c).rc c = 1 ∧
      Heap.Exclusive (mutate Hc p (parentAfter op i) i c) p
```

The child's admitted body and the returned-handle accounting are implicit; the conclusion is the exclusive branch and the parent's count.

7 Runtime model and abort impossibility

The model's count is a ghost count; the runtime's is the one that decides. The runtime model is a separate state machine with its own heap, counts, persistence, guarded allocator, check and two array-set dispositions. The structural correspondence preserves refs, cells, counts, persistence, fields and the cursor; from it the model's exclusivity implies the runtime's, the check returns, and the write mutates. The abort branch is unreachable under the correspondence and `EntryInv`*.*

The definitions are ordinary Lean structures. The correspondence contains no exclusivity or disposition; those are derived from the preserved counts, persistence and fields. A guarded allocator refuses a dirty cursor, so the copy branch cannot overwrite a live cell.

Claim 10 (No abort).

```
theorem checkExclusive_no_abort {H : Heap} {R : RHeap} {r : Ref}
    (hc : Corr H R) (hex : H.Exclusive r) (r' : Ref) :
    checkExclusive R r ≠ Runtime.CheckOutcome.abort r'
```

```
theorem setAt_inPlace_of_entryInv
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hC : Corr H R) (hcell : H.cells root = some o)
    (hR : R.cells root = some o') (hin : i < o.fields.size) :
    ∃ H',
      setAt R root i (.scalar vn) = some (.inPlace H' root) ∧
      Corr (inPlaceOutcome root i vn o H).heap H' ∧
      (inPlaceOutcome root i vn o H).target = some root ∧
      ((H'.cells root).bind fun o => o.field? i) = some (.scalar vn)'
```

Claim 11 (Emitted-semantics abort impossibility).

```
def PrefixAborts (out : Option PrefixOutcome) : Prop :=
  ∃ o, out = some o ∧ o.fault ≠ none

theorem prefix_abort_impossible
    (hproj : projectPrefix code = some p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    ¬ PrefixAborts (PrefixOutcome.ofExec assign p (execAST assign p code s))
```

The layout and the remaining entry premises are implicit; this is the emitted-semantics form of the no-abort result.

8 The principal guarantee and its contracts

The model, the recognised LCNF and the runtime model are separate semantics. Their composition at the flat recognised write is `principal_write`*:*

```
theorem principal_write
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hacc : OwnersAccounted (Runner.initCtx root)) (vars : List Var)
    (hcount : CountFidelity H (Runner.initCtx root) vars)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size)
    (hc : Corr H R) (ho' : R.cells root = some o') :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
      PrefixOutcome.ofModel (Ownership.run (Prefix.toModel assign p) env H) =
        inPlaceOutcome root i v o H ∧
      ∃ H', Runtime.setAt R root i (.scalar v) =
          some (.inPlace H' root) ∧
        Corr (inPlaceOutcome root i v o H).heap H' ∧
        (inPlaceOutcome root i v o H).target = some root ∧
        ((H'.cells root).bind fun o => o.field? i) = some (.scalar v)
```

The proof is the conjunction of `recognized_write_in_place` *and* `Runtime.setAt_inPlace_of_entryInv`*; the accounting hypotheses are not used by those proofs. They are the contracts that make the model's count one the physical count one;* `alias_not_accounted` *shows why the environment side cannot be omitted. The theorem is flat: the nested path is the separate* `index_path_simulates_model`*, whose* `PathContract` *is discharged for a finite level list by* `PathRecursion` *and leaves the leaf contract and N10.*
The full type of `principal_write` *is in Appendix A; the model and LCNF claims are stated separately in their sections.*

9 Empirical alignment

The model is only as good as its match to the compiler. A compiled probe compares the model's predicates with `lean_is_exclusive` *and address identity on a corpus: a unique array, a live alias, the alias released before the write, a persistent literal and a closure-held array. The closed-pure-array case is the important negative: a pure closed array is lifted to a* `___closed` *constant read through* `lean_obj_once`*, so it is shared and*

Contract	Proposition	Provider	Status or remaining obligation
`EntryInv`	unique tree, client separation, root completeness	client entry	`initEntryInv` for fresh arrays; arbitrary heap is a client obligation
`Agrees`	heap/environment/code agreement and annotations	client	supplied per declaration
A1–A12	primitive, layout and call signatures	implementation assumption	documented; not proved of the C runtime
N1–N12	nested path and call contracts	implementation/leaf	N10 and the leaf body remain
`OwnersAccounted`	environment owner count at zero and one	client obligation	predicate and preservation lemmas; checker take-out/`dec` preconditions not enforced
`CountFidelity`	heap count = incoming edges + owning variables	implementation assumption	not proved; the residual runtime fidelity
`PathContract`	finite-path child contract	client	tail discharged by induction; leaf remains
`Corr`	runtime model correspondence	proved relation	C semantics and address identity remain assumptions
linked runtime	`lean_is_exclusive`, `lean_array_uset`	empirical	gate and differential corpus, not a proof
source correspondence	Lean source to emitted LCNF	assumption	not attempted

Table 4: Contracts of the principal guarantee, classified by provider.

the write copies even though the source looks unique. The gate also binds the pinned artifacts, the IR shapes, the generated C and the runtime header. The corpus is evidence, not an exhaustive equivalence.

Case	Checker/model	Runtime count	Buffer	Result	Config
unique array	accepted	exclusive	in place	preserved	off; on passes
live alias (count two)	rejected	shared	copy	alias 0	off; on aborts
alias released before write	runtime-only	exclusive	in place	preserved	off; on passes
persistent literal	not certified	shared	copy	preserved	off; on aborts
closure-held array	not certified	shared	copy	preserved	off; on aborts
closed-pure-array	not certified	shared	copy	preserved	off
nested `setAt`	model/LCNF accepted	model in place	pair allocation	C/IR gate	no linked differential

Table 5: Observed alignment for the flat probe and the nested LCNF/IR gate.

Table 5 separates the checker result from the runtime disposition. The linked probe covers the flat primitives; the nested sequence is bound at the generated-C/IR level, not by a linked differential. The closed-pure-array case is the negative control: a pure closed array is lifted to a `___closed` *constant read through* `lean_obj_once`*, so it is shared and the write copies even though the source looks unique. The corpus is evidence, not an exhaustive equivalence.*

The cost claim is conditional. A single alias need not force a copy on every iteration: after the first copy the array can be unique, so the quadratic effect requires sustained sharing at each update. No timing is claimed. The temporary extraction pair may allocate even when the certified array write is in place; no-copy certification is not an allocation-free execution theorem.

10 Limits and non-claims

Non-claim 1.
The boundary is explicit. The C semantics of the primitives is not formalised; replay-to-backend identity (A10) and address identity remain assumptions; the emitted recursive callee and continuation (N10) are not captured; the model's flat environment cannot express the emitted per-invocation index; the leaf body contract and several heap-shape facts remain supplied; free/cascade, persistence, concurrency and callbacks are outside the fragment; and the source correspondence is not attempted.

The accounting contracts are not consequences of `Inv`. The environment side (`OwnersAccounted`) has a kernel countermodel and explicit `bindOwned`/`dec` preconditions that the checker does not yet enforce; the heap side (`CountFidelity`) is an implementation assumption about the runtime's count. The recognition claim is scoped to the accepted grammar and the mutation corpus, not a universal rejection theorem for every token-preserving near miss. The nested linked runtime is not measured; only the flat probe is.

11 Related work

Huisinga's 2023 uniqueness analysis for a model of Lean's IR [6] is the closest predecessor; Lean's `Array.markLinear` *[12] is a dynamic mark/abort facility, and translation validation [15] checks a compiler run rather than proving a static discipline. Reference counting and reuse: Perceus and Counting Immutable Beans [16, 19], and Koka's effect typing [13]. Linear and uniqueness disciplines: Wadler's linear types [20], Linear Haskell [2], Clean's uniqueness [1], alias types [18], ownership types [4] and Cyclone [7]. Program logics: separation logic [17], Iris [9], RustBelt [8] and Verus [11]. Verified compilation and kernels: CompCert [14], CakeML [10], Lean 4 [5] and Lean4Lean [3]. The contrast is the one drawn in Table 1: these systems put linearity in the type system, the borrow checker or the compiler, while this development checks the emitted fragment and proves the connection in Lean.*

12 Conclusion

The development proves the count-one discipline at the model, the recognised LCNF and the runtime model, under named contracts, and composes the flat guarantee as `principal_write`*. It measures the linked primitive on a corpus and gates the pinned evidence. The accounting contracts and the C semantics, the replay/backend identity, the emitted callee and the source correspondence remain the boundary. The next step is either a verified runtime model or a stronger backend gate; the development proves soundness for the accepted fragment, not completeness of the analysis.*

A Theorem map and artifacts

The table lists the headline declarations and their status. "Proved" means a kernel-checked theorem; "contracts" and "leaf/N10" name the assumptions under which the theorem is stated; "measured" means the compilation gate and the corpus, not a proof.

A.1 Headline declarations

`Ownership.check_sound`
`Ownership/Soundness.lean` — *proved; model.*

`Ownership.Entry.checkN_sound_of_entryInv`
`Ownership/Entry.lean` — *proved; model,* `EntryInv` *and* `Agrees`*.*

`OwnershipLcnf.project_sound`
`OwnershipLcnf/Semantics.lean` — *proved under the primitive contracts.*

`OwnershipLcnf.recognized_write_in_place`
`OwnershipLcnf/Semantics.lean` — *proved under the primitive contracts.*

OwnershipLcnf.index_path_simulates_model
OwnershipLcnf/PathRecursion.lean — proved with `PathContract`*; the finite tail is discharged; leaf and N10 remain.*

Ownership.CountedPair.pair_accounting
Ownership/CountedPair.lean — proved; restricted free.

Ownership.PutBack.putBack_exclusive_of_wellFormed
Ownership/PutBack.lean — proved; returned-handle accounting remains.

OwnershipLcnf.Runtime.checkExclusive_no_abort
OwnershipLcnf/RuntimeModel.lean — proved in the runtime model.

OwnershipLcnf.Runtime.setAt_inPlace_of_entryInv
OwnershipLcnf/RuntimeModel.lean — proved in the runtime model.

OwnershipLcnf.prefix_abort_impossible
OwnershipLcnf/NoAbort.lean — proved under A6/A8.

OwnershipLcnf.principal_write
OwnershipLcnf/Principal.lean — proved by composing the flat LCNF and runtime-model claims; carries the accounting contracts.

Ownership.OwnersAccounted, Ownership.CountFidelity
Ownership/Accounting.lean — definitions; preservation lemmas for `setUs`*,* `move`*,* `inc`*,* `bindOwned`*,* `dec` *with explicit preconditions.*

Ownership.alias_not_accounted
Ownership/Accounting.lean — kernel countermodel; the raw `Inv` *admits two owners at multiplicity one.*

differential corpus, runtime-binding gate
runtime-bridge, scripts/check_runtime_binding.sh — measured.

A.2 Full principal type

```
theorem principal_write
    {code : Lean.Compiler.LCNF.Code Lean.Compiler.LCNF.Purity.impure}
    {p : Prefix} {assign : Lean.FVarId → Var}
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (s : Machine) (env : Env) (H : Heap)
    (root : Ref) (client : List Ref) (i v : Nat) (o : Obj)
    (hEntry : Entry.EntryInv H env (Runner.initCtx root) root client)
    (hacc : OwnersAccounted (Runner.initCtx root))
    (vars : List Var)
    (hcount : CountFidelity H (Runner.initCtx root) vars)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size)
    {R : Runtime.RHeap} {o' : Runtime.RObj}
    (hc : Runtime.Corr H R) (ho' : R.cells root = some o') :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
      PrefixOutcome.ofModel (Ownership.run (Prefix.toModel assign p) env H) =
        inPlaceOutcome root i v o H ∧
        ∃ H',
          Runtime.setAt R root i (Val.scalar v) =
              some (Runtime.SetOutcome.inPlace H' root) ∧
            Runtime.Corr (inPlaceOutcome root i v o H).heap H' ∧
              (inPlaceOutcome root i v o H).target = some root ∧
                ((H'.cells root).bind fun o => o.field? i) = some (Val.scalar v)
```

A.3 Reproduction

Pinned toolchain: `leanprover/lean4:v4.35.0-rc3`*. Pinned sibling:* `lean-examples`*, clean, at*

`66e64334a1c968d829903b8aec8d22f7e3b9fd3f`. *The paper's local TeX build is not part of the artifact.*

```
nix develop -c lake build Ownership
nix develop -c lake build OwnershipLcnf
nix develop -c lake build OwnershipTest
nix develop -c lake env lean scripts/check_axioms.lean
nix develop -c bash scripts/check_artifacts.sh
nix develop -c bash scripts/check_runtime_binding.sh
```

Observed on 1 November 2026 at the base revision `fcab2f39e9bf086cbb5817c947dadc1f68c8c605`*: all three Lean targets build; the artifact gate reports "artifacts match the pinned sibling and toolchain"; the runtime gate reports "pinned artifacts, IR shapes, C/runtime primitives and probe modes agree". The probe exits zero for* `separate-exact`*,* `separate-spare`*,* `inline-io`*,* `sealed`*,* `inline-exact`*,* `closed-case` *and* `differential` *with the check off, and for* `separate-exact`*,* `inline-io` *and* `sealed` *with the check on;* `inline-exact` *and* `closed-case` *exit one with the check on. The axiom script reports only* `propext`*,* `Classical.choice` *and* `Quot.sound` *for the headline theorems and* `propext` *for* `alias_not_accounted`*.*

The C semantics, replay-to-backend identity, address identity, free/cascade and the emitted recursive callee remain outside the artifact. The runtime gate binds presence and observed behaviour, not a refinement of the C semantics.

References

[1] Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. Mathematical Structures in Computer Science*, 6(6):579–612, 1996.*

[2] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In Principles of Programming Languages (POPL)*, 2018.*

[3] Mario Carneiro. Lean4lean: Towards a formalized proof checker for lean 4, 2024. Preprint.

[4] Dave Clarke, John Potter, and James Noble. Ownership types for flexible alias protection. In Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 1998.*

[5] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In Automated Deduction – CADE-28*, volume 12699 of* LNCS*. Springer, 2021.*

[6] Marc Huisinga. Static uniqueness analysis for the lean 4 theorem prover. Master's thesis, Karlsruhe Institute of Technology, 2023. Master's thesis.

[7] Trevor Jim, J. Gregory Morrisett, Dan Grossman, Michael W. Hicks, James Cheney, and Yanling Wang. Cyclone: A safe dialect of c. In USENIX Annual Technical Conference*, 2002.*

[8] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: Securing the foundations of the rust programming language. In Principles of Programming Languages (POPL)*, 2018.*

[9] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In Principles of Programming Languages (POPL)*, 2015.*

[10] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In Principles of Programming Languages (POPL)*, 2014.*

[11] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. In Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2023.*

[12] Lean FRO. Array.marklinear, 2025. Lean API documentation.

[13] Daan Leijen. Type directed compilation of row-typed algebraic effects. In Principles of Programming Languages (POPL)*, 2017.*

[14] Xavier Leroy. Formal verification of a realistic compiler. Communications of the ACM*, 52(7), 2009.*

[15] George C. Necula. Translation validation for an optimizing compiler. In Programming Language Design and Implementation (PLDI)*, 2000.*

[16] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In Programming Language Design and Implementation (PLDI)*, 2021.*

[17] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In Logic in Computer Science (LICS)*, 2002.*

[18] Frederick Smith, David Walker, and J. Gregory Morrisett. Alias types. In European Symposium on Programming (ESOP)*, 2000.*

[19] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In Implementation and Application of Functional Languages (IFL)*, 2019.*

[20] Philip Wadler. Linear types can change the world! Programming Concepts and Methods*, 1990.*